[Biometria] Projekt 2 - Dokumentacja

Maciej Andrzejewski, Ludwik Madej

Spis treści

1 Opis projektu **2**
1.1 Wykorzystane biblioteki . . . 2
1.2 Instrukcja uruchomienia . . . 2

2 Opis metod **3**
2.1 Wczytanie obrazu (`load_grayscale_image`) . . . 3
2.2 Detekcja źrenicy . . . 3
2.2.1 Progowanie i analiza kołowości (`detect_pupil_geometry`) . . . 3
2.2.2 Uściślenie geometrii (`refine_pupil_roi`) . . . 3
2.3 Detekcja tęczówki (`detect_iris_geometry`) . . . 4
2.4 Iteracyjna detekcja geometrii (`get_eye_geometry`) . . . 4
2.4.1 Przeszukiwanie siatki parametrów (`find_optimal_eye_geometry`) . . . 5
2.5 Ocena jakości geometrii (`Evaluator`) . . . 5
2.6 Rozwinięcie tęczówki (`unroll_iris`) . . . 5
2.7 Kodowanie tęczówki (`encode_iris`) . . . 5
2.8 Porównanie kodów tęczówki (`compare_iris_codes`) . . . 6

3 Prezentacja wyników **6**
3.1 Zbiór danych . . . 6
3.2 Wyniki detekcji geometrii . . . 6
3.3 Kodowanie całego zbioru . . . 7
3.4 Rozkład odległości Hamminga . . . 7
3.5 Metryki systemu biometrycznego . . . 8
3.6 Empirycznie uzyskane progi decyzyjne . . . 9
3.7 Przykładowe porównanie pojedynczej (losowej) pary . . . 9
3.8 Test ręczny z wizualizacją . . . 9

4 Wnioski **9**
4.1 Jakość detekcji geometrii . . . 9
4.2 Analiza wyników biometrycznych . . . 10
4.3 Podsumowanie . . . 10

1 Opis projektu

Projekt stanowi implementację algorytmu rozpoznawania tęczówki metodą Daugmana. Całość została zaimplementowana w języku Python w oparciu o skrypty zorganizowane w strukturę pakietu oraz przetestowana w notatnikach Jupyter Notebook. Nie implementowano interfejsu graficznego do obsługi kodu.

Rozwiązanie obejmuje następujące etapy:

1. Wczytanie zdjęcia oka oraz jego konwersja do skali szarości
2. Detekcja obszaru źrenicy oraz tęczówki oka
3. Rozwinięcie pierścienia tęczówki do postaci prostokątnej
4. Zakodowanie rozwiniętej tęczówki do postaci ciągu bitów (algorytm Daugmana)
5. Porównanie dwóch kodów przy pomocy odległości Hamminga

Kluczowym założeniem projektu była ręczna implementacja wszystkich kluczowych aspektów algorytmów biometrycznych, dlatego też użyte biblioteki wykorzystano wyłącznie do niskopoziomowych operacji na macierzach oraz obrazach.

1.1 Wykorzystane biblioteki

- NumPy - do obliczeń numerycznych: operacji macierzowych, transformacji między układami współrzędnych, operacji splotowych, obliczeń statystycznych. Dzięki operacjom zwektoryzowanym możliwe jest bardzo szybkie przeprowadzenie wyżej wymienionych operacji
- OpenCV - do pracy na obrazach: wczytania pliku, konwersji do odpowiedniej postaci, zastosowania operacji morfologicznych, wyrównania histogramu, inpaintigu odblasków oraz innych zagadnień
- Matplotlib - do wyświetlania obrazów (oczu, wykrytych źrenic i tęczówek, rozwiniętej tęczówki) oraz wyników (kod tęczówki oraz wykresy ewaluacji metody)

1.2 Instrukcja uruchomienia

Program wymaga środowiska Python 3.10 oraz zainstalowanych zależności zgodnych z plikiem `requirements.txt` dołączonym do repozytorium. Zalecane jest utworzenie wirtualnego środowiska, aby uniknąć konfliktów z innymi pakietami zainstalowanymi w systemie.

Aby móc odtworzyć otrzymane w notatnikach wyniki należy wykonać następujące kroki:

1. Pobrać repozytorium projektu i przejść do jego głównego katalogu:

   ```
   cd Projekt2
   ```

2. Utworzyć i aktywować wirtualne środowisko Python 3.10, a następnie zainstalować wymagane zależności:

   ```
   python -m venv .venv
   .venv\Scripts\activate        # Windows
   source venv/bin/activate     # Linux / macOS
   pip install -r requirements.txt
   ```

3. Przy pomocy aplikacji umożliwiającej pracę z notatnikami Jupyter otworzyć wybrany przez siebie notatnik, wybrać plik `.venv\Scripts\python.exe` jako kernel notatnika oraz uruchomić komórki po kolei.

2 Opis metod

2.1 Wczytanie obrazu (`load_grayscale_image`)

Funkcja wczytuje obraz (domyślnie w formacie `.bmp`) i konwertuje go do skali szarości. Funkcja sprawdza również istnienie pliku przed wywołaniem wczytania, ponieważ biblioteka OpenCV w przypadku błędu zwraca `None` zamiast wyrzucić wyjątek.

2.2 Detekcja źrenicy

Detekcja źrenicy przebiega dwuetapowo: wstępna detekcja na całym obrazie (`detect_pupil_geometry`), a następnie dokładniejsze dopasowanie w wyciętym regionie zainteresowania (`refine_pupil_roi`). Wyniki obu kroków są łączone ważoną średnią, gdzie waga każdego wyniku zależy od ciemności wykrytej powierzchni (ciemniejsza detekcja - bardziej prawdopodobna prawdziwa źrenica).

2.2.1 Progowanie i analiza kołowości (`detect_pupil_geometry`)

Próg binaryzacji wyznaczany jest wzorem:

$$T = \frac{P}{\ln P},$$

gdzie P jest średnią jasnością obrazu. Formuła ta daje niski próg dla ciemnych obrazów (tylko najciemniejsze piksele są kandydatami na źrenicę) i nieco wyższy dla jasnych. Następnie obraz jest progowany (po zastosowaniu inwersji), a operacje morfologiczne (otwarcie jądrem 3×3 i zamknięcie jądrem 7×7) usuwają szum i scalają pobliskie obszary.

Spośród wykrytych konturów wybierany jest ten o najwyższej kołowości:

$$\text{kołowość} = \frac{4\pi A}{P^2},$$

gdzie A to pole powierzchni konturu, a P jego obwód (dla idealnego koła kołowość wynosi 1). Kontur, aby być brany pod uwagę, musi przekroczyć próg kołowości 0.5 oraz minimalną powierzchnię 100 px. Centrum i promień źrenicy wyznaczane są przez minimalne koło otaczające wybrany kontur.

2.2.2 Uściślenie geometrii (`refine_pupil_roi`)

W wyciętym regionie ($2r \times 2r$ wokół centrum z poprzedniego kroku) usuwane są odblaski rogówki przez inpainting pikseli jaśniejszych od progu 200. Następnie obraz jest wygładzany filtrem Gaussa i skanowany w trzech kierunkach (prawo, lewo, dół) w poszukiwaniu krawędzi źrenicy - miejsca, gdzie jasność przekracza próg $I_{\text{centrum}} + 35$. Na podstawie pozycji lewej i prawej krawędzi wyznaczany jest nowy środek i promień zgodnie ze wzorami:

$$x_{\text{new}} = \frac{x_L + x_R}{2}, \quad r_{\text{new}} = \frac{x_R - x_L}{2}.$$

2.3 Detekcja tęczówki (`detect_iris_geometry`)

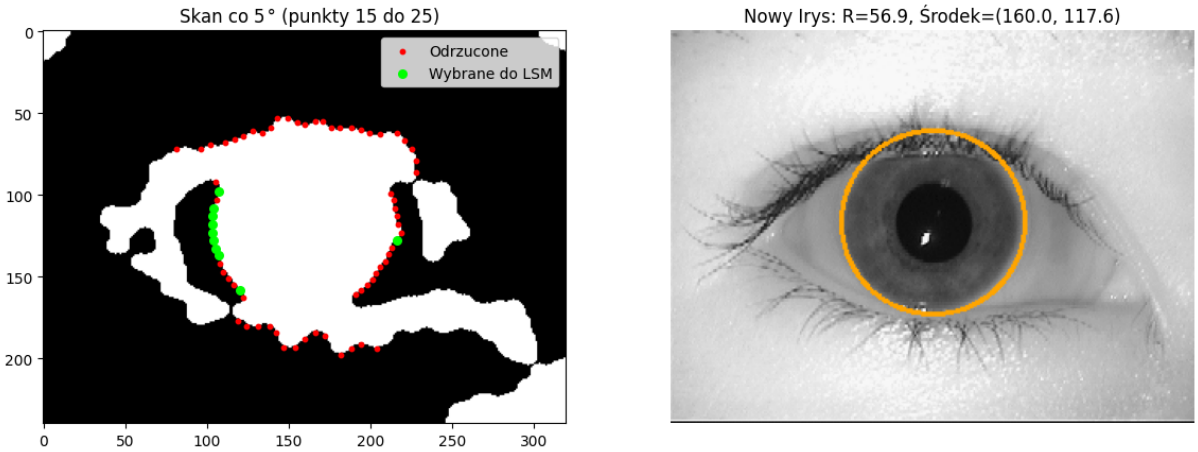
Detekcja tęczówki również przebiega w dwóch podobnych etapach:

1. Na obrazie o wzmocnionym kontraście wykonywane jest progowanie 20. percentylem jasności, a wynik oczyszczany operacjami morfologicznymi (zamknięcie i otwarcie jądrem 11×11). Skanowanie promieniowe odbywa się wzdłuż 72 kierunków co 5 stopni ze względu na różną orientację zdjęć. Dla każdego kierunku rejestrowany jest punkt, w którym maska binarna zmienia się z 1 na 0 (koniec ciemnego obszaru - granica tęczówki z tłem).

   Zebrane punkty są sortowane po odległości od centrum i środkowe (po odrzuceniu punktów zbyt bliskich i zbyt dalekich) trafiają do algorytmu dopasowania okręgu metodą najmniejszych kwadratów (LSM). Równanie okręgu $x^2 + y^2 = 2x_c \cdot x + 2y_c \cdot y + (x_c^2 + y_c^2 - r^2)$ jest przepisywane jako układ liniowy $\mathbf{Ax} = \mathbf{b}$ i rozwiązywane przez `np.linalg.lstsq`:

$$\mathbf{A} = \begin{bmatrix} 2x_1 & 2y_1 & 1 \\ \vdots & \vdots & \vdots \\ 2x_n & 2y_n & 1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} x_1^2 + y_1^2 \\ \vdots \\ x_n^2 + y_n^2 \end{bmatrix}.$$

2. W powierzchni z 20% marginesem wokół okręgu z kroku 1 skanowanie powtarzane jest w węższym zakresie odległości $[0.8\, r_i;\ 1.3\, r_i]$. Daje to bardziej precyzyjny wynik, wolny od fałszywych krawędzi odległych od tęczówki.



Rysunek 1: Wykres pokazujący otrzymywanie punktów, do których dopasowywany zostaje okrąg metodą MNK. Zakres punktów jest hiperparametrem, od którego w znacznym stopniu zależy jakość dopasowania

2.4 Iteracyjna detekcja geometrii (`get_eye_geometry`)

Funkcja, która iteracyjnie dokonuje detekcji źrenicy i tęczówki w następujących krokach:

1. Normalizacja jasności: jeśli średnia jasność > 170, obraz jest przyciemniany. W przeciwnym razie rozciągany jest do pełnego zakresu jasności.

2. W każdej z iteracji kontrast jest podwyższany multiplikacyjnie, a rozmiar obrazu skalowany w dół z uwzględnieniem wykrytej w poprzedniej iteracji geometrii.

3. Po każdej iteracji uruchamiana jest detekcja źrenicy i tęczówki. Następnie współrzędne przeliczane są z powrotem do układu oryginalnego.

Mniejszy obraz ułatwia wykrycie źrenicy bez wpływu szumów drobnych tekstur. Gdy detekcja tęczówki zwróci promień zbyt bliski promieniu źrenicy, stosowane jest zabezpieczenie: $r_i = 2.5 \cdot r_p$.

2.4.1 Przeszukiwanie siatki parametrów (`find_optimal_eye_geometry`)

Gdy standardowa detekcja daje niestabilne wyniki, dostępna jest funkcja `find_optimal_eye_geometry`, która przeszukuje siatkę hiperparametrów w celu znalezienia najlepszego rozwiązania. Każda konfiguracja jest oceniana przez klasę `Evaluator`, a wynik końcowy to ważona średnia geometria z 5 najlepszych konfiguracji.

2.5 Ocena jakości geometrii (`Evaluator`)

Klasa `Evaluator` oblicza ważoną sumę ośmiu niezależnych miar jakości wykrytej geometrii:

```
- Daugman na źrenicy (0.25): najważniejszy - ostra, wyraźna granica źrenicy
- Kontrast (0.15): źrenica musi być ciemniejsza od tęczówki
- Wyrównanie krawędzi źrenicy (0.15): gradient musi być radialny
- Daugman na tęczówce (0.15): ostra granica zewnętrzna
- Geometria (0.10): źrenica wewnątrz tęczówki
- Wyrównanie krawędzi tęczówki (0.10): gradient zewnętrzny
- Wyciek (0.05): brak ciemnych pikseli za granicą źrenicy
- Spójność (0.05): jednorodność tęczówki
```

Wynik Daugmana dla okręgu (x_c, y_c, r) mierzy spójność gradientu radialnego wzdłuż próbkowanych punktów obwodu:

$$S_{\text{Daugman}} = \frac{\text{mean}(\nabla_r)}{\text{std}(\nabla_r)},$$

gdzie ∇_r to rzuty wektora gradientu Sobela na kierunek radialny. Wynik jest normalizowany przez tanh do przedziału $[0, 1]$ przed wyliczeniem ważonej sumy.

2.6 Rozwinięcie tęczówki (`unroll_iris`)

Pierścień tęczówki jest przekształcany do prostokąta o rozmiarze 60×360 pikseli metodą *rubber-sheet model* zaproponowaną przez Daugmana. Każdy piksel prostokąta (r, θ) odpowiada punktowi tęczówki:

$$x(r, \theta) = (1 - r)\, x_p(\theta) + r\, x_i(\theta), \tag{1}$$

$$y(r, \theta) = (1 - r)\, y_p(\theta) + r\, y_i(\theta), \tag{2}$$

gdzie $r \in [0, 1]$ jest współrzędną radialną ($r = 0$ - granica źrenicy, $r = 1$ - granica tęczówki), $\theta \in [0, 2\pi]$ kątem, a $x_p(\theta) = x_p + r_p \cos\theta$ punktem na granicy źrenicy:

$$x_p(\theta) = x_p + r_p \cos\theta, \qquad y_p(\theta) = y_p + r_p \sin\theta,$$

$$x_i(\theta) = x_i + r_i \cos\theta, \qquad y_i(\theta) = y_i + r_i \sin\theta.$$

Mapy odwzorowania (x, y) obliczane są wektorowo i przekazywane do funkcji `cv2.remap` z interpolacją bikubiczną, która daje lepsze wygładzenie tekstury niż liniowa.

Dzięki tej transformacji obraz tęczówki jest znormalizowany niezależnie od rozmiaru źrenicy i tęczówki - ta sama osoba sfotografowana z różnych odległości da podobny prostokąt.

2.7 Kodowanie tęczówki (`encode_iris`)

Rozwinięty obraz tęczówki (60×360 pikseli) jest kodowany jako kod binarny metodą Daugmana z użyciem falki Gabora:

1. Obraz dzielony jest na 8 poziomych pasm radialnych.

2. Pomijane są kolumny, które zawierają piksele powieki, a nie tęczówki. Z pozostałych kolumn wybieranych jest 128 równomiernie rozmieszczonych pozycji kątowych.
3. Dla każdego pasma b wiersze pasma zwijane są do jednego sygnału 1D przez ważoną średnią gaussowską (środkowe wiersze mają większy wkład), a sygnał jest normalizowany do zerowej średniej i wariancji równej 1.
4. Sygnał splatany jest z parą jąder falki Gabora:

$$g_{\mathrm{re}}(x) = e^{-x^2/(2\sigma^2)} \cos(2\pi f x), \quad g_{\mathrm{im}}(x) = e^{-x^2/(2\sigma^2)} \sin(2\pi f x),$$

   gdzie $\sigma = \frac{1}{2}\pi f$, a $f = 1.5$.
5. Znak wyniku tego splotu koduje 2 bity (części rzeczywistej i urojonej) dla każdej pozycji:

$$b^{\mathrm{re}}_{i,j} = \mathbf{1}[\mathrm{conv_re}[i,j] \geq 0], \quad b^{\mathrm{im}}_{i,j} = \mathbf{1}[\mathrm{conv_im}[i,j] \geq 0].$$

Bity są układane w następujący sposób: dla każdej ze 128 pozycji kątowych zapisywany jest blok $8 \times 2 = 16$ bitów $[b^{\mathrm{re}}_0, b^{\mathrm{im}}_0, b^{\mathrm{re}}_1, \ldots, b^{\mathrm{im}}_7]$. Taki układ sprawia, że przesunięcie o jedną pozycję kątową odpowiada dokładnie przesunięciu tablicy bitów o 16 pozycji, co upraszcza eliminację wpływu rotacji w trakcie porównywania dwóch kodów.

Otrzymany kod zawiera łącznie $8 \times 128 \times 2 = 2048$ bitów.

Parametr f (częstotliwość falki Gabora) wpływa na to, jakie struktury tęczówki są kodowane: małe f uchwyci grubą strukturę radialną, duże f - drobne tekstury. Zakres użyteczny wynosi w przybliżeniu $[0.6;\ \pi]$, natomiast wartości spoza tego zakresu dają zbyt długie lub zbyt krótkie jądra.

2.8 Porównanie kodów tęczówki (`compare_iris_codes`)

Dwa kody tęczówki porównywane są przy pomocy znormalizowanej odległości Hamminga:

$$\mathrm{HD}(code_1, code_2) = \frac{1}{N} \sum_{k=1}^{N} \big(code_{1,k} == code_{2,k}\big) \in [0,1].$$

Dla dwóch losowych, niezależnych kodów odległość Hamminga powinna wynosić ≈ 0.5.

Aby skompensować możliwą rotację oka pomiędzy zdjęciami, sprawdzane są przesunięcia w pewnym zakresie na tablicy bitów.

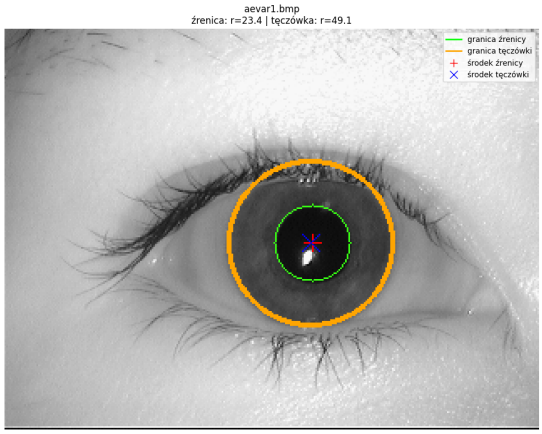
3 Prezentacja wyników

3.1 Zbiór danych

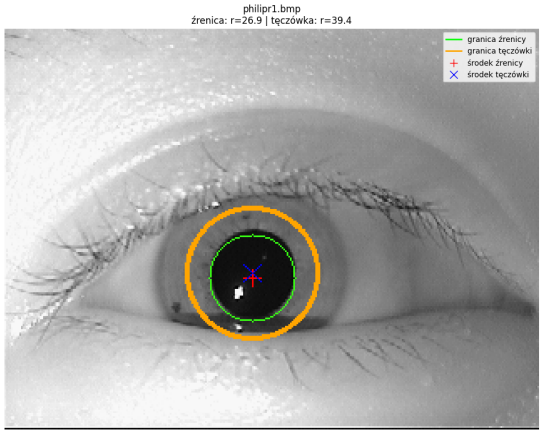
Eksperymenty przeprowadzono na bazie **MMU Iris Database** zawierającej 450 obrazów oczu 45 osób (po 5 zdjęć lewego i prawego oka). Obrazy mają rozdzielczość 320×240 pikseli i są zapisane w formacie BMP.

3.2 Wyniki detekcji geometrii

Funkcja `find_optimal_eye_geometry` z przeszukiwaniem siatki hiperparametrów przetestowana została na 10 wybranych obrazach z bazy. We wszystkich 10 przypadkach detekcja zakończyła się sukcesem, jednak w kilku sytuacjach okrąg otaczający tęczówkę był zbyt mały lub zbyt duży.



Poprawne dopasowanie okręgu tęczówki



Niepoprawne dopasowanie okręgu tęczówki

Rysunek 2: Porównanie wyników detekcji tęczówki

3.3 Kodowanie całego zbioru

Kodowanie uruchomione na wszystkich 450 obrazach z bazy dało następujące wyniki:

Tabela 1: Wyniki kodowania całego zbioru MMU Iris Database.

Miara	Wartość
Zakodowane obrazy	437 / 450
Nieudane kodowanie	13
Skuteczność kodowania	97.1%

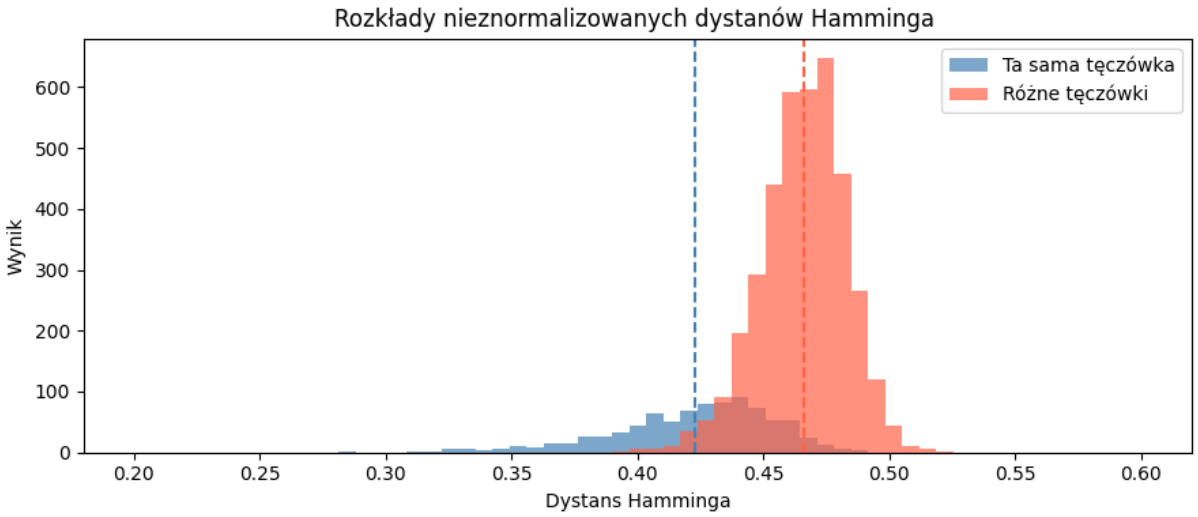
13 obrazów zostało odrzuconych z powodu błędnej geometrii (np. promień zbyt mały lub zbyt duży w stosunku do rozmiarów oka). Warto wziąć pod uwagę, że przypadki zakodowania tęczówki dopasowanej niedokładnie (przykład w poprzedniej sekcji) zostały tutaj uznane za poprawnie zakodowane. W rzeczywistości więc współczynnik skutecznego zakodowania jest niższy.

3.4 Rozkład odległości Hamminga

Na podstawie 437 zakodowanych obrazów wygenerowane zostały pary do weryfikacji:

Tabela 2: Statystyki odległości Hamminga dla par genuine i impostor.

Typ par	Liczba par	Średnia HD	Odch. std. HD
Ta sama tęczówka	864	0.4226	0.0317
Różne tęczówki	3872	0.4658	0.0168



Rysunek 3: Rozkład nieznormalizowanego dystansu Hamminga dla par tych samych tęczówek oraz par różnych tęczówek

Pary dla tej samej tęczówki tworzono ze zdjęć tego samego oka tej samej osoby (różne ujęcia), natomiast pary różnych tęczówek z pierwszych ujęć różnych osób.

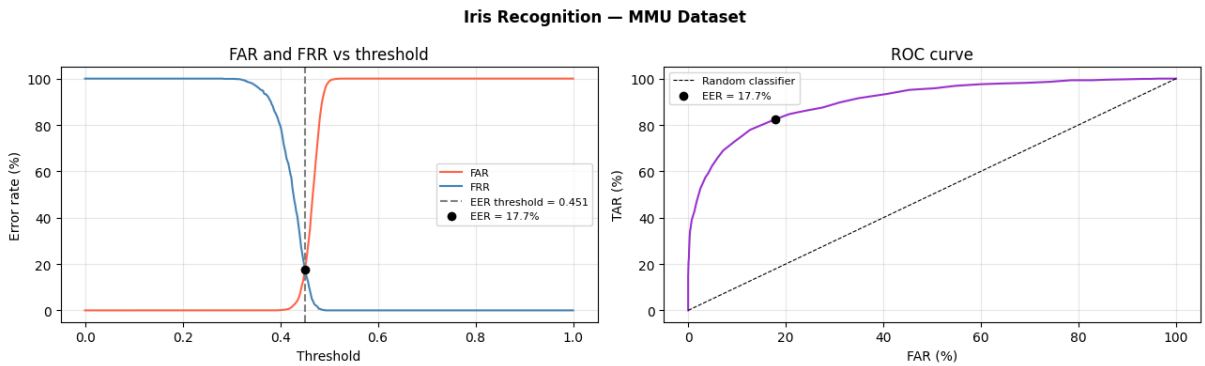
3.5 Metryki systemu biometrycznego

Na podstawie utworzonych par kodów dokonano porównań oraz ich oceny metrykami FAR, FRR oraz EER. Wyniki dla punktu EER zamieszczono w poniższej tabeli.

Tabela 3: Wyniki oceny systemu na zbiorze MMU.

Metryka	Wartość
Próg przy EER	0.4509
Equal Error Rate (EER)	17.69%
FAR przy EER	17.79%
FRR przy EER	17.59%

EER wyznaczono jako punkt przecięcia krzywych FAR i FRR w funkcji progu binaryzacji odległości Hamminga. Na poniższych wykresach przedstawiono wizualizację wyznacoznego punkt EER.



Rysunek 4: Wizualizacja punktu EER

3.6 Empirycznie uzyskane progi decyzyjne

Na podstawie powyższych wyników ustalono następujące progi decyzyjne (dla zbioru MMU Iris Database):

Tabela 4: Progi decyzyjne dla zbioru MMU Iris Database.

Zakres HD	Interpretacja
< 0.4	Ta sama tęczówka
0.4 - 0.47	Niepewny wynik
> 0.47	Różne tęczówki

3.7 Przykładowe porównanie pojedynczej (losowej) pary

Przykładowe porównanie dwóch kodów dało:

- Para tej samej tęczówki: HD = 0.3530
- Para dwóch różnych tęczówek: HD = 0.4761

Wyniki potwierdzają, że w przypadku dobrze odczytanych i zakodowanych tęczówek system potrafi rozróżnić tę samą i różne tęczówki.

3.8 Test ręczny z wizualizacją

Notatnik `6_manual_test_with_vis.ipynb` umożliwia interaktywne przetestowanie pipeline'u na dowolnych dwóch zdjęciach. W przypadku problemu z automatyczną detekcją granic źrenicy i tęczówki ta może zostać przeprowadzona przy pomocy interaktywnego widgetu (szczegóły opisane w notatniku).

4 Wnioski

4.1 Jakość detekcji geometrii

Detekcja źrenicy oparta na progu $P/\ln P$ sprawdziła się w przypadku obrazów z bazy MMU Iris Dataset (standardowe warunki, stałe oświetlenie, źrenica wyraźnie ciemniejsza od tęczówki). Problemy pojawiają się jednak w przypadku:

- czarnego tła zdjęcia - progowanie wyodrębnia tło zamiast źrenicy,
- silnych odblasków - inpainting w `refine_pupil_roi` częściowo łagodzi problem, jednak duże odblaski mogą zaburzyć skanowanie jasności,
- słabego kontrastu - iteracyjne podbijanie kontrastu w `get_eye_geometry` zwiększa stabilność, lecz przy bardzo jasnych obrazach może prowadzić do nasycenia i utraty struktury.

Wszystkie powyższe problemy można w pewien sposób zaadresować, jednak jest to dodatkowa praca, która najczęściej nie przynosi stabilnych, powtarzalnych rezultatów dla różnych zdjęć w obrębie problemu.

Detekcja tęczówki jest trudniejsza ze względu na zmienny kształt granicy (powieki, rzęsy) i niższy kontrast. Wyrównanie histogramu skutecznie poprawia lokalny kontrast, jednak przy silnie zacienionych obrzeżach tęczówki zastosowana metoda najmniejszych kwadratów (LSM) może dopasować zbyt mały lub zbyt duży okrąg.

4.2 Analiza wyników biometrycznych

Uzyskany EER na poziomie $17,69\%$ jest znacznie gorszy niż standardowy wynik dla algorytmu Daugmana ($< 1\%$). Główne przyczyny takiego stanu rzeczy to między innymi:

1. Błędy geometrii - nawet niewielkie odchylenie środka okręgu źrenicy lub tęczówki powoduje wycięcie innego fragmentu tęczówki i degradację kodu tęczówki.
2. Nachodząca powieka - wyłączenie pewnych kolumn rozwiniętej tęczówki z analizy jest tylko statycznym przybliżeniem. Prawdziwa granica powieki jest indywidualna i różni się między ujęciami.

4.3 Podsumowanie

Implementacja poprawnie realizuje algorytm Daugmana - jest w stanie stwierdzić czy dwa zdjęcia przedstawiają to samo oko, czy jednak te dwa zdjęcia to obrazy dwojga różnych oczu. Głównym ograniczeniem jest tutaj jakość detekcji źrenicy i tęczówki - nieprecyzyjne wyznaczenie granic obu fragmentów oka skutkuje znaczącym pogorszeniem jakości zakodowania tęczówki, a w konsekwencji zaburzeniem obiektu służącego do bezpośredniego porównywania dwóch tęczówek.